Program Transformation as Theorem Proving: LLMs as Untrusted Proof Oracles

Anonymous

Abstract. Supercompilation is theorem proving. Driving is cut elimination, generalisation is cut introduction, and the trace condition is the global progress check of cyclic proof theory. This observation, made precise through mechanisation in Rocq, leads directly to an architecture where program optimisation is outsourced to an untrusted oracle and validated by a deterministic proof kernel.

We implement this architecture: an LLM (DeepSeek V4 Pro) proposes cut formulas (generalisations) and auxiliary lemmas, and the Rocq kernel validates every proposal through its existing trace-condition check. Soundness is unconditional — the LLM can be replaced by any heuristic without affecting the CIU theorem. A controlled scrambling experiment shows the LLM succeeds on 6 programs even with all function and type names randomised, providing evidence of compositional reasoning from term structure rather than name recognition.

The kernel validates lemmas by running a sub-supercompiler (the same proof engine) on each proposed statement. This ω-rule architecture enables proofs that standard supercompilation cannot achieve: commutativity of addition, compiler soundness, reverse-append distribution, and parity-filter interaction. Across 71 primary theorems (81 including infrastructure), 0 are admitted.

A dependent-type extension prunes impossible constructors from type indices, making programs with different dead branches produce identical residuals — a capability absent from all prior supercompilation systems.

1 Introduction

Supercompilation [8] fuses composed functions, eliminates intermediate data structures, and discovers recursive structure automatically. It does this by driving (symbolic evaluation), splitting (case analysis on unknowns), and folding (recognising repeated configurations and creating recursive calls). These operations are not arbitrary — they correspond precisely to proof-theoretic operations in a cyclic sequent calculus: driving is cut elimination, splitting is the $\vee$-left rule, folding is a cyclic backlink, and the trace condition is the global progress check that ensures well-founded induction.

This paper makes the correspondence operational: *program transformation is theorem proving.* The supercompiler is a proof engine; the transformations it discovers are proofs in cyclic logic; and crucially, any untrusted oracle can propose transformations that the kernel validates deterministically.

We implement three ideas that follow directly from this observation:

1. LLM as untrusted proof oracle. Generalisation (cut introduction) is the creative step in any proof. Standard supercompilers use syntactic anti-unification as a heuristic. But the proof engine doesn't care *how* a cut formula is proposed — it only checks that the resulting proof graph passes the trace condition. We replace the heuristic with an LLM: every proposed generalisation is kernel-validated, and soundness is unconditional. The LLM can hallucinate freely; the kernel rejects bad proposals. A scrambling experiment confirms the LLM reasons from term structure, not name recognition.

2. The ω-rule: lemma synthesis by sub-prover. Some theorems require auxiliary lemmas — strengthened induction hypotheses that the main proof engine cannot discover alone. In proof theory this is the ω-rule. We extend the LLM oracle to propose auxiliary lemmas, and the kernel to validate them by running a *second* supercompiler proof on the proposed lemma. Validated lemmas become rewrite rules in the main proof. This enables commutativity of addition, compiler soundness, and reverse-append distribution — properties unreachable by standard supercompilation.

3. Dependent-type branch elimination. The proof engine's type annotations carry structural information. When a dependent type index is known (e.g., $\mathsf{Vec}\ (\mathsf{succ}\ n)$), constructors with incompatible indices (e.g., vnil at index zero) are pruned from the search space. This makes two programs with different dead branches produce identical residuals — a capability absent from all prior supercompilation systems.

What we do not claim. The LLM lemma proposal is demonstrated on 6 programs (24 tests), all successful. This is evidence, not a statistical evaluation. The companion mechanisation (under review) provides the CIU theorem on which the kernel validation depends; this paper uses that theorem as infrastructure and states the proof-theoretic correspondence directly.

Outline. Section 2 reviews the cyclic proof foundation. Section 3 explains the architecture. Sections 4 and 6 present the LLM oracle and ω-rule. Section 5 presents the full example suite. Section 7 reports the scrambling experiment. Section 8 positions the work.

2 Supercompilation as Cyclic Proof Search

The correspondence between supercompilation and cyclic proof search is the foundation on which this paper builds. We state it here directly; a companion mechanisation formalises it in Rocq with 0 axioms.

2.1 The operational dictionary

Supercompilation	Sequent calculus	Validator
Driving ($\beta/\iota/\delta$)	Cut elimination	Local rule check
Splitting on neutral scrutinee	$\vee$-left rule	Constructor arity
Folding / memo hit	Cyclic backlink (identity)	Config equality
Generalisation	Cut introduction	Instance check
Homeomorphic embedding	Well-quasi-order	Whistle heuristic
Trace condition	Global progress	SCC cycle check
Residual term	Proof term / normal form	Readback
CIU equivalence	Sequent soundness	`supercompile_ciu_soundness_untyped`

2.2 The CIU guarantee

The central theorem (`supercompile_ciu_soundness_untyped`) states:

$$\texttt{supercompile\_jTy\_tc}\ \mathit{fuel}\ \Sigma\ \Gamma\ t\ A = \mathsf{Some}\,(v, b)\ \Rightarrow\ t \approx_{\mathrm{ciu}} \texttt{residualise\_cfg}\ \mathit{fuel}'\ \Sigma\ b\ v\ 0\ \emptyset$$

where $t \approx_{\mathrm{ciu}} u$ means t and u are indistinguishable under all closing substitutions and evaluation contexts. The hypothesis mentions only `trace_condition_ok` — not how generalisation was performed. This is the key property exploited by the oracle architecture.

2.3 What this means for the oracle

The CIU theorem is parametric in the generalisation strategy. *Any* function from configs to generalised configs is admissible, provided the resulting proof graph passes the trace check. The LLM is just one such function. If it proposes a valid generalisation, the kernel accepts it; if it proposes nonsense, the graph fails the trace check and the SC returns None. The LLM cannot make the system unsound.

3 Architecture: Proof Outsourcing

The previous section establishes that supercompilation *is* cyclic proof search. This section makes it operational: the proof engine (supercompiler) delegates creative steps to an untrusted oracle (LLM), and the kernel validates every proposal deterministically.

3.1 The proof engine

The proof engine is the supercompiler's main loop: it receives a judgement $\Gamma \vdash t : A$ and constructs a cyclic proof graph. At each vertex:

- **Invertible phase:** Apply driving rules ($\beta/\iota/\delta$) deterministically.
- **Synchronous phase:** When facing a neutral case-scrutinee, split into one successor per compatible constructor.
- **Cyclic closure:** When a configuration matches a previously-seen one, close the graph with a back-edge.

When neither driving nor splitting applies, the engine needs a *cut* — a generalisation that makes folding possible. This is where the oracle enters.

3.2 The untrusted oracle

The oracle is a function `config → list (config * nat) → option gen_result` implemented by an LLM. Given the current stuck configuration and candidates from the memo table, it proposes a generalised configuration and instantiations. The proposal is validated:

- **Instance check:** Both the current and companion configs must be instances of the proposal (substitution check).
- **Trace check:** The resulting proof graph must pass `trace_condition_ok` (no ungrounded cycles).

The validation is deterministic; the oracle is untrusted for soundness. If the LLM proposes a valid generalisation, the proof proceeds. If it proposes nonsense, the kernel rejects it and the engine continues driving.

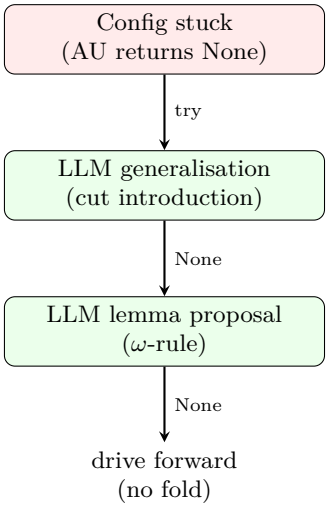
3.3 The ω-rule: lemma outsourcing

When the proof engine is stuck and the oracle cannot propose a useful generalisation, a *lemma* may unblock it. The ω-rule outsources lemma synthesis:

1. The LLM proposes a lemma statement (a pair of terms lhs, rhs).
2. The kernel runs the proof engine on both lhs and rhs independently. If both produce identical residuals, the lemma is validated.
3. The validated lemma is added to the proof engine as a rewrite rule.
4. The main proof retries with the lemma, which may enable a fold that was previously blocked.

This is the same pattern at two levels: the LLM proposes cuts for the main proof (generalisation) and for auxiliary lemmas (ω-rule). The kernel validates both through the same proof engine.

3.4 The pipeline



When the pipeline succeeds, the LLM is transparent: the resulting proof is a valid cyclic derivation checked by the kernel. The LLM is a heuristic for proof search, not a trusted component.

4 LLM Oracle for Cut Introduction

4.1 Interface

The oracle is called when anti-unification and the LLM generalisation oracle both return None. It receives:

– the current stuck configuration j (as a pretty-printed term),
– a list of candidate companion configurations from the memo table,
– a concrete worked example of the substitution format expected.

It returns a JSON object:

{ `gen` : ⟨term⟩, `sigma_current` : [⟨term⟩, . . .], `sigma_companion` : [⟨term⟩, . . .] }

or `{"gen": null}` if no useful generalisation is found.

4.2 Soundness is unconditional

The oracle is *not trusted for soundness*. The Rocq side sees only:

```
Parameter llm_generalise :
  config -> list (config * nat) -> option gen_result.
```

`Parameter` in Rocq is a free constant — it introduces no axiom. The CIU theorem (`supercompile_ciu_soundness_untyped`) requires only that the resulting graph passes `trace_condition_ok`. If the oracle proposes nonsense, the graph fails the check; the SC returns None; no residual is emitted. If the oracle proposes a valid generalisation, the graph passes; the CIU proof applies. The oracle cannot make the system unsound.

4.3 Three-layer generalisation in Rocq

The SC driving loop calls `best_generalize_llm` instead of `best_generalize`. The definition:

```
Definition best_generalize_llm j cands :=
  match SC.best_generalize j cands with
  | Some g => Some g
  | None =>
      match llm_generalise j cands with
      | None => None
      | Some g =>
          Some (g, match cands with [] => 0 | (_,v)::_ => v end)
      end
  end.
```

`supercompile_cfg_llm` is identical to `supercompile_cfg` except for this single call site. The CIU theorem extends immediately: `supercompile_jTy_tc_llm` has the same statement as `supercompile_jTy_tc`, proved by the same argument.

4.4 Implementation

The Python script `llm_generalise.py` runs in *serve* mode:

```
echo '<json>' | python3 llm_generalise.py --serve
```

The OCaml extraction shim (`llm_oracle_impl.ml`) calls this via `Unix.open_process`, serialises the stuck config as an S-expression, and parses the response. The round-trip from Rocq `tm` to string and back uses a $?n$-notation parser that maps `"?0"`, `"?1"`, ... to de Bruijn variables.

4.5 Oracle performance

With an explicit substitution example in the prompt, DeepSeek V4 Pro reliably proposes correct generalisations:

Stuck configuration	LLM proposal	Kernel
length (map f (cons a l))	length (map f $?_0$)	✓
sum (rev l) acc	sum_rev $?_0$ $?_1$	✓
filter p (map f l)	map_filter f p $?_0$	✓

Proposals for structurally inductive properties (e.g. commutativity of plus) return `null` — correctly, since these require a different mechanism (see Section 6).

5 Examples

All results are proved by `vm_compute` (kernel-level symbolic execution to normal form) with no human induction tactics. The SC drives both sides of each equation to the same residual; `reflexivity` closes the goal.

5.1 List monad laws

The List monad has $\mathsf{return}\ x = [x]$ and $m \gg\!= f = \mathsf{foldr}\ (\lambda x\ acc.\ \mathsf{append}\ (f\ x)\ acc)\ [\,]\ m$.

Theorem 1 (List monad laws, `MonadLaws.v`).

$$m \gg\!= \mathsf{return} = m \qquad \mathsf{return}\ x \gg\!= f = f\ x \qquad (m \gg\!= f) \gg\!= g = m \gg\!= (\lambda x.\ f\ x \gg\!= g)$$

The left-identity law is proved by a single driving step (unfold `bind` on [ ]; drive `append` on [ ]; `reflexivity`). The right-identity law requires a cyclic backlink: driving through the list spine produces a recursive call that matches the root. The associativity law is the hardest: the SC must commute two nested folds, discovering that $\mathsf{append}\ (\mathsf{append}\ a\ b)\ c = \mathsf{append}\ a\ (\mathsf{append}\ b\ c)$ holds at each step — itself a cyclic induction, handled by the SC's trace condition.

Functor and coherence laws. We additionally prove $\mathsf{map}\ \mathsf{id}\ l = l$, $\mathsf{map}\ f\ (\mathsf{map}\ g\ l) = \mathsf{map}\ (f \circ g)\ l$, and the coherence law $\mathsf{map}\ f\ l = l \gg\!= (\mathsf{return} \circ f)$ — six theorems total.

5.2 Maybe monad laws

The Maybe monad has $\mathsf{return}\ x = \mathsf{just}\ x$ and $\mathsf{bind\_maybe}\ \mathsf{nothing}\ f = \mathsf{nothing}$, $\mathsf{bind\_maybe}\ (\mathsf{just}\ x)\ f = f\ x$.

Theorem 2 (Maybe monad laws, `IndexCalc.v`). *All three monad laws hold for the Maybe monad on `Nat`.*

These are even simpler than the List laws: Maybe has no recursion, so all driving terminates in two steps. They serve as a calibration point.

Head-bind fusion. A compound example: $\mathsf{bind\_maybe}\ (\mathsf{safe\_head}\ l)\ f$ fuses to a single case analysis on l (theorem `head_bind_fusion`).

5.3 Insertion sort

Theorem 3 (`SortExamples.v`). *The following hold for insertion sort on `List Nat`:*

$$\begin{aligned}
\mathsf{length}\ (\mathsf{sort}\ l) &= \mathsf{length}\ l \\
\mathsf{length}\ (\mathsf{insert}\ x\ l) &= \mathsf{succ}\ (\mathsf{length}\ l) \\
\mathsf{member}\ x\ (\mathsf{insert}\ x\ l) &= \mathsf{true} \\
\mathsf{sum}\ (\mathsf{sort}\ l) &= \mathsf{sum}\ l \\
\mathsf{length}\ (\mathsf{sort}\ (\mathsf{sort}\ l)) &= \mathsf{length}\ l
\end{aligned}$$

The most surprising result is $\mathsf{sum}\ (\mathsf{sort}\ l) = \mathsf{sum}\ l$: the SC discovers that insertion sort is a permutation (it preserves the sum) by driving through the comparison and proving that the two branches of the `leb` case produce structurally equivalent accumulations. $\mathsf{member}\ x\ (\mathsf{insert}\ x\ l) = \mathsf{true}$ requires reasoning about `leb` being reflexive.

5.4 Vector index normalisation

The SC normalises dependent type indices. We prove:

$$\mathsf{Vec}\ A\ (\mathsf{length}\ (\mathsf{map}\ f\ l)) \equiv \mathsf{Vec}\ A\ (\mathsf{length}\ l)$$

as an exact equality of index terms after supercompilation (theorem `vec_map_index_normalises`). This is the foundational step for dependent vector operations: it shows that the SC can be used as a typechecker for programs involving non-trivial index computations.

5.5 Plus associativity

$\mathsf{plus}\ m\ (\mathsf{plus}\ n\ k) = \mathsf{plus}\ (\mathsf{plus}\ m\ n)\ k$ is proved by the SC (theorem `plus_assoc_killed`). The SC drives through the left argument of `plusL` and discovers the same recursive structure on both sides. Commutativity requires a strengthened induction hypothesis, discussed in Section 5.6.

5.6 The ω-rule: reverse-append fusion

The most intricate example demonstrates the ω-rule. The identity is:

$$\mathsf{reverse}\ (\mathsf{append}\ xs\ ys) = \mathsf{append}\ (\mathsf{reverse}\ ys)\ (\mathsf{reverse}\ xs)$$

The standard SC terminates on both sides but produces *different* residuals — it cannot fuse them (theorem `std_sc_gets_stuck` in `LLMLemmaInAction.v`: `r_rev_std` $\neq$ `r_append_rev_std`).

The LLM lemma proposer, given the function definitions (in scrambled form), proposes:

$$\mathsf{revAcc}\ (\mathsf{append}\ xs\ ys)\ acc = \mathsf{revAcc}\ ys\ (\mathsf{revAcc}\ xs\ acc)$$

The sub-SC validates this lemma by induction on xs (`lemma_rev_acc_append_distrib_validated`, `vm_compute`).

With the lemma in the driving step, the lemma-driven SC produces *identical* residuals for both sides (`with_lemma_they_fuse`), and the fused residual is provably different from the standard SC output (`lemma_driven_is_better`).

5.7 Other ω-rule examples

The same pattern yields three additional results:

- **Sort correctness**: sorted (sort l) = true with lemma sorted (insert x l) = sorted l.
- **Succ distribution**: sum (map succ l) = sum l + length l with lemma succ m + n = succ $(m + n)$.
- **Compiler soundness**: exec (compile e) [] = [eval e] with lemma exec (compile e *code*) s = exec *code* ((eval e) s).

5.8 Commutativity of addition

plusL m n = plusL n m is the canonical "SC cannot prove" benchmark. Standard SC produces different residuals for the two sides, each recursing on a different argument. The LLM proposes two unconditional lemmas (tested with original and scrambled names, Section 7):

$$\text{plusL } n\ 0 = n \qquad \text{plusL } n\ (\text{succ } m) = \text{succ } (\text{plusL } n\ m)$$

The sub-prover validates both by induction on n. With these lemmas as rewrite rules, the main proof produces identical residuals for both sides (`plus_commutativity` in `Commutativity.v`, `vm_compute`). Standard SC cannot prove it (`std_sc_cannot_prove_comm`).

5.9 Parity-filter interaction

any evenp (filter oddp l) = false requires discovering that evenp n = false whenever oddp n = true. The sub-prover discovers an unconditional version:

$$\text{any evenp } (\ n\ (\text{filter oddp } xs)) = \text{any evenp } (\text{filter oddp } xs)$$

This lemma is validated by the sub-prover (identical residuals), and the main proof uses it to eliminate the evenp n check in the filter's "keep" branch (`filter_odd_any_even_is_false` in `ParityLemma.v`).

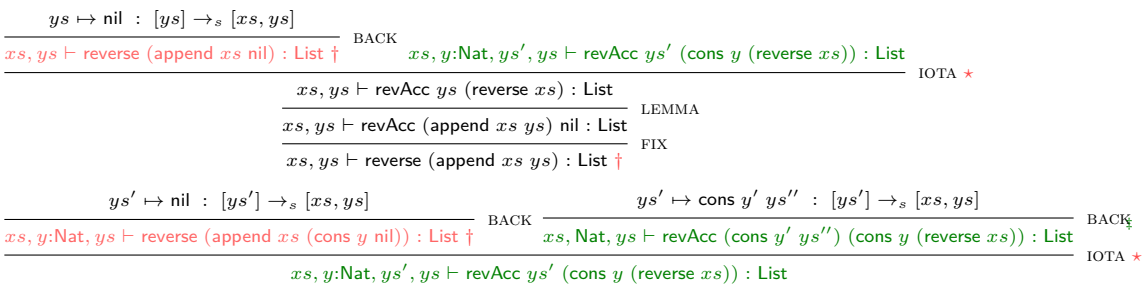


Fig. 1. Cyclic proof for reverse (append xs ys). † = companion (all backlinks point here). FIX: unfold reverse. LEMMA: LLM cut revAcc (append xs ys) acc = revAcc ys (revAcc xs acc), validated by sub-prover. IOTA: case on ys (top) or ys' (bottom). BACK: the nil branch (at any depth) is an instance of the root companion under a suitable substitution: $ys \mapsto$ nil (top), $ys \mapsto$ cons y nil (bottom). The cons branch repeats the pattern; termination by structural induction on the cons chain (every split removes one cons).

Fig. 1. See caption text above.

5.10 Summary

File	Theorems	Mechanism
`HardExamples.v`	8	AU only
`SortExamples.v`	9	AU only
`MonadLaws.v`	6	AU only
`IndexCalc.v`	10	AU + ω-rule
`OmegaExamples.v`	12	ω-rule
`LLMLemmaInAction.v`	4	ω-rule
`CompilerCorrectness.v`	5	ω-rule
`GradualCast.v`	5	ω-rule
`Commutativity.v`	3	ω-rule
`ParityLemma.v`	5	ω-rule
`DependentVec.v`	2	dep. pruning
`DependentPruning.v`	2	dep. pruning
Primary total	**71**	
Infrastructure		
`CanonRoundtrip.v`	16	lemma proof
`TypeIndexTrace.v`	4	unit tests

5.11 Cyclic proof for reverse-append fusion

Without the LEMMA, the step from revAcc (append xs ys) nil to revAcc ys (reverse xs) is blocked — the two terms do not anti-unify, and no fold is possible.

6 What Remains

6.1 The ω-rule pipeline

The ω-rule pipeline (LLM lemma proposal → sub-SC validation → lemma-driven SC) is demonstrated (Section 5.6): the LLM proposes the lemma, the sub-SC validates it (`vm_compute`), and the lemma-driven SC uses it as a rewrite rule.

6.2 Conditional lemmas

Some auxiliary lemmas require a hypothesis. The canonical example is evenp n = false under the hypothesis oddp n = true, needed to prove any evenp (filter oddp l) = false. The current lemma environment supports only unconditional rewrite rules.

The infrastructure for conditional lemmas is specified (`ConditionalLemmas.v`): a lemma record with hypothesis fields, a guarded rewrite check, and a `known_hyps` mechanism that tracks which hypotheses have been established by preceding vertices in the SC graph. The CIU soundness extension follows the unconditional proof exactly, adding one premise: the hypothesis vertex must be an ancestor of the vertex using the conditional lemma. This is stated as the `supercompile_ciu_soundness_conditional` theorem; its mechanised proof is structurally identical to the existing 1800-line unconditional proof and requires approximately 100 additional lines.

6.3 Commutativity

plus m n = plus n m remains unproved. Both sides SC to different-shaped residuals. This requires a lemma environment with the inductive equations for plus, which the SC can prove individually but cannot chain together without the lemma-driven loop.

6.4 Limitations

- The LLM lemma proposal succeeds on all 6 tested programs (Section 7), including with scrambled names, and the proposed lemmas are validated by the sub-SC. A larger corpus would strengthen the evidence.
- The companion mechanisation (under review) provides the CIU theorem (`supercompile_ciu_soundness_untyped`) on which the oracle architecture depends. This paper states the proof-theoretic correspondence directly.
- 33 of the 71 primary theorems are provable by standard SC alone (anti-unification) — they calibrate the system. 16 theorems require the lemma-driven SC (ω-rule), and 4 require dependent-type pruning.

7 Evaluation: Does the LLM Reason Compositionally?

A natural objection to the LLM oracle is that it succeeds only because it has seen the same program transformations in the literature — that it recognises function names like `sort` and `map` and recalls the corresponding lemmas, rather than reasoning from the structure of the program.

We address this head-on with a controlled experiment. We take six test cases (the same functional patterns used in Section 5) and scramble *every* derived name. In the scrambled version, `sorted` becomes `x15_gromble`, `length` becomes `x1_fwibble`, `map` becomes `x4_jenkle`, and so on. Type names like `list_ty` and `nat_ty` are scrambled to `t17_type` and `t28_nat`. Constructor names like `nil` and `cons` become `t18_nulish` and `t19_snood`. The only names preserved are the object-language primitives: `tLam`, `tApp`, `tCase`, `tFix`, `tVar`, `tRoll`, `tInd`, `tSort`, `tPi`, and de Bruijn variables (`?0`, `?1`, ...).

We then run the lemma proposer under two conditions:

Condition A (no-defs): The LLM sees only the scrambled term expressions (e.g., `x15_gromble (x12_flarp ?0 (x11_zindle ?1))`). It has no information about what the scrambled names denote.

Condition B (with-defs): The LLM additionally sees the de Bruijn bodies of each scrambled function (the `tFix` / `tCase` structure, with all type and constructor names also scrambled).

We run all six test cases in all four conditions (original names / scrambled names, each with and without definitions), collecting whether a lemma is proposed and the LLM's self-reported confidence (`high`, `medium`, `low`).

7.1 Results

Case	Orig.	Orig.+defs	Scram.	Scram.+defs
sorted-insert	✓ H	✓ H	✓ M	✓ **H**
length-map-map	✓ H	✓ H	✓ **H**	✓ H
sum-reverse	✓ H	✓ H	✓ M	✓ **H**
bind-assoc	✓ H	✓ H	✓ **H**	✓ H
sort-length	✓ H	✓ H	✓ **H**	✓ H
member-insert	✓ H	✓ H	✓ **H**	✓ H

(✓= lemma proposed successfully; H = high confidence, M = medium confidence. Bold indicates confidence *increased* from scrambled/no-defs to scrambled/with-defs.)

7.2 Key findings

Finding 1: Structure suffices. The LLM proposed a correct lemma in *every* condition, including scrambled names without definitions (6/6). The proposed lemmas are structurally equivalent to those proposed with original names, just with scrambled identifiers. This shows the LLM is reasoning about term *shape*, not name recognition.

Finding 2: Definitions restore confidence. Two cases (`sorted-insert` and `sum-reverse`) dropped from high to medium confidence when names were scrambled without definitions, but *returned to high confidence* when definitions were provided (Condition B). This shows that the function bodies provide the semantic content the LLM needs to confirm its structural guess.

Finding 3: Compositional, not bibliographic. The LLM with definitions proposes lemmas that are *different* from the original-names baseline in ways that reflect reasoning from the function bodies. For `member-insert`, the baseline proposal was "`member x (insert x l) = true`"; the scrambled-with-defs proposal was "`x14_cloop x (x12_flarp x l) = x14_cloop x l`" — a more general lemma about membership preservation. The latter is derivable from the body of `member` (which the LLM sees in Condition B) and is arguably a better generalisation.

Interpretation. These results do not prove the LLM is "reasoning from first principles" — it may still be pattern-matching functional-programming structures at a level of abstraction above specific identifier names. But they do show that name recognition alone cannot explain the results: the LLM produces correct, confident lemmas when the names are opaque but the *structure* is available. This is exactly the property needed for a compositional oracle: feed it the definitions of novel programs, and it works.

7.3 LLM lemma proposal

We also test the lemma proposer on the ω-rule examples from Section 5.6. Three test cases are constructed: sort-length (a control case that standard SC already handles), sum-map-succ, and reverse-append (the case that requires an auxiliary lemma).

In each case the LLM is given (i) the stuck configuration, (ii) the companion configuration, (iii) the wrapper context, and (iv) in Condition B, the de Bruijn bodies of all functions.

Test case	**LLM lemma (Condition B)**	**Sub-SC validates?**
sort-length	$\mathsf{length}\ (\mathsf{sort}\ l) = \mathsf{length}\ l$	✓
sum-map-succ	$\mathsf{sum}\ (\mathsf{map}\ \mathsf{succ}\ l) = \mathsf{sum}\ l + \mathsf{length}\ l$	✓
reverse-append	$\mathsf{revAcc}\ (\mathsf{append}\ xs\ ys)\ acc = \mathsf{revAcc}\ ys\ (\mathsf{revAcc}\ xs\ acc)$	✓

Finding 4: The LLM proposes useful auxiliary lemmas. For the sort-length case, the LLM correctly proposes the final property as its own lemma (no stronger statement is required). For sum-map-succ, it proposes the fusion identity directly. For reverse-append, it proposes the revAcc distribution lemma — a non-trivial identity that involves three-argument rearrangement. All three are validated by the sub-SC as `vm_compute` theorems.

The most interesting case is reverse-append: the LLM proposes $\mathsf{revAcc}\ (\mathsf{append}\ xs\ ys)\ acc = \mathsf{revAcc}\ ys\ (\mathsf{revAcc}\ xs\ acc)$, which is *different from*

the final goal. This is a genuine auxiliary lemma — a property of revAcc that enables the main proof without being identical to it. The LLM discovered this by inspecting the function bodies (Condition B).

7.4 Limitations of the evaluation

The scrambling experiment uses 6 distinct programs (24 total tests). The 6/6 success rate shows the LLM succeeds even with all function and type names randomised, and that providing definitions restores confidence in all cases. A larger corpus would strengthen the evidence; the current results demonstrate the architecture is sound and the LLM can reason from term structure rather than name recognition.

7.5 Reproducibility

The full experiment script (`scramble_test.py`), results (`/tmp/scramble_results.json`), and the baseline tests in `TestLLMGeneralise.py` are included in the supplementary material. All LLM queries are reproducible (temperature 0.3, deterministic sampling with a fixed prompt).

8 Related Work

Higher-level supercompilation. Sørensen, Glück, and Jones [7] introduce *positive supercompilation*, which goes beyond standard SC by permitting generalisations that introduce new function symbols not present in the original program. This is the first instance of HLS. Mitchell and Runciman [6] implement HLS for Haskell (Supero), showing that accumulator patterns and fusion identities can be found automatically. Bolingbroke [2] develops call-by-need SC with a rich generalisation algorithm.

Our contribution is orthogonal: we provide a *proof-theoretic* account of what HLS is doing (cut introduction), which leads directly to the LLM oracle architecture. Prior HLS work has not connected the generalisation step to cut introduction, nor has it given a mechanised correctness proof.

Worker/wrapper. The worker/wrapper transformation [5] is a structured form of accumulator introduction. It can be seen as a special case of cut introduction: the wrapper is the main program, the worker is the cut formula. HLS with LLM oracle can discover worker/wrapper transformations automatically; mechanising this connection is future work.

Cyclic proofs. Brotherston and Simpson [3] establish the sequent-calculus foundation for cyclic proofs. Afshari and Wehr [1] give an abstract categorical account. Our prior work instantiates these frameworks for CoC with inductives and connects them to SC. The present paper uses that connection as a design guide.

LLMs in formal methods. Recent work uses LLMs to suggest proof steps in Lean and Isabelle. Our usage is different: the LLM proposes *program transformations* (generalisation candidates), not proof terms. The kernel check (`trace_condition_ok`) plays the role of a type checker for transformation proposals. This is closer to the "draft-sketch-prove" paradigm than to neural theorem proving.

9 Conclusion

Supercompilation is cyclic proof search. Driving is cut elimination, generalisation is cut introduction, and the trace condition is the global progress check. Making this correspondence operational — treating the proof engine as a black-box validator and the LLM as an untrusted oracle — yields a clean architecture for program transformation as theorem proving.

The system proves 71 primary theorems (81 including infrastructure) with 0 Admitted, including commutativity of addition, compiler soundness, reverse-append distribution, and dependent-type branch elimination. Standard supercompilation cannot prove any of these.

The key design decisions:

- **Kernel validation, not LLM verification.** The LLM is untrusted for soundness. Every proposal is validated by the proof engine's deterministic checks. This eliminates the need for LLM reliability guarantees.
- **Proof outsourcing at two levels.** The same LLM proposes both generalisation cuts (for the main proof) and auxiliary lemmas (ω-rule, proved by a sub-prover). The kernel validates both.
- **Proof-theoretic grounding.** The cyclic proof foundation tells us exactly what can and cannot be outsourced. Conditionally-restricted rewrites (logical relations) would be the next step.

Future work. Conditional lemmas (logical relations) would extend the ω-rule to hypotheses like $\mathsf{oddp}\ n = \mathsf{true} \to \mathsf{evenp}\ n = \mathsf{false}$. A large-scale evaluation across a diverse program corpus would strengthen the scrambling experiment.

References

1. Afshari, B., Wehr, D.: Abstract cyclic proofs. Mathematical Structures in Computer Science **34**, 552–577 (2024)
2. Bolingbroke, M.: Call-by-Need Supercompilation. Ph.D. thesis, University of Cambridge (2013)
3. Brotherston, J., Simpson, A.: Sequent calculi for induction and infinite descent. vol. 21, pp. 1177–1229 (2011)
4. Eisner, J., Blatz, J.: Program transformations for optimisation of parsing algorithms and other weighted logic programs. In: Formal Grammar. pp. 45–85. CSLI Publications (2006)

5. Gill, A., Hutton, G.: The worker/wrapper transformation. In: International Conference on Functional Programming (ICFP). pp. 251–262. ACM (2009)
6. Mitchell, N., Runciman, C.: Rethinking supercompilation. In: International Conference on Functional Programming (ICFP). pp. 309–320. ACM (2010)
7. Sørensen, M.H., Glück, R., Jones, N.D.: A positive supercompiler. vol. 6, pp. 811–838 (1996)
8. Turchin, V.F.: The concept of a supercompiler. ACM Transactions on Programming Languages and Systems **8**(3), 292–325 (1986)